

# DW\_pulse\_sync

## Dual Clock Pulse Synchronizer

Version, STAR, and myDesignWare Subscriptions: [IP Directory](#)

### Features and Benefits

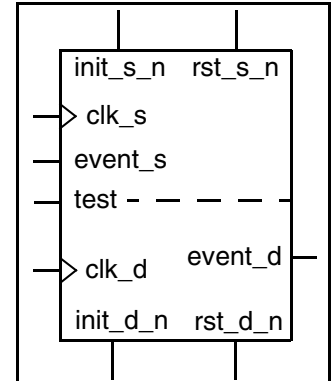
- Fully tested cross clock domain
- Fully parameterized
- Able to use both positive and negative clock edge for sending clock domain
- Provides for both combinatorial and registered output, via parameter

### Revision History

### Description

DW\_pulse\_sync provides a low-risk method for transmitting single clock cycle pulses between two different clock domains. This component uses clock-domain-crossing techniques to safely transfer pulses between logic operating on different clocks. The *pulse\_mode* parameter determines the type of pulse that is transmitted; choices are toggle, rising edge, falling edge, or the default single source clock cycle pulse.

Simulation models are available in Verilog and VHDL. Synthesizable source is available in Verilog.



**Table 1-1 Pin Description**

| Pin Name | Width | Direction | Function                       |
|----------|-------|-----------|--------------------------------|
| clk_s    | 1     | Input     | Source clock                   |
| rst_s_n  | 1     | Input     | Asynchronous source reset      |
| init_s_n | 1     | Input     | Synchronous source reset       |
| event_s  | 1     | Input     | Input pulse                    |
| clk_d    | 1     | Input     | Destination clock              |
| rst_d_n  | 1     | Input     | Asynchronous destination reset |
| init_d_n | 1     | Input     | Synchronous destination reset  |
| event_d  | 1     | Output    | Output pulse                   |
| test     | 1     | Input     | Scan test mode select input    |

**Table 1-2 Parameter Description**

| Parameter   | Values               | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| reg_event   | 0 or 1<br>Default: 1 | <ul style="list-style-type: none"> <li>0: No register on output</li> <li>1: Register <code>event_d</code> output</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                |
| f_sync_type | 0 to 4<br>Default: 2 | <ul style="list-style-type: none"> <li>0: Single clock design <code>clk_d = clk_s</code></li> <li>1: Neg-edge to pos-edge sync</li> <li>2: Pos-edge to pos-edge sync</li> <li>3: Three pos-edge flops in dest domain</li> <li>4: Four pos-edge flops in dest domain</li> </ul>                                                                                                                                                                                                                                             |
| tst_mode    | 0 to 2<br>Default: 0 | <ul style="list-style-type: none"> <li>0: No test latch insertion</li> <li>1: Hold latch using neg-edge flop</li> <li>2: Hold latch using active low latch</li> </ul>                                                                                                                                                                                                                                                                                                                                                      |
| verif_en    | 0 to 4<br>Default: 1 | Verify enable control <ul style="list-style-type: none"> <li>0: No sampling errors inserted</li> <li>1: Sampling errors are randomly inserted with 0 or up to 1 destination clock cycle delays</li> <li>2: Sampling errors are randomly inserted with 0, 0.5, 1, or 1.5 destination clock cycle delays</li> <li>3: Sampling errors are randomly inserted with 0, 1, 2, or 3 destination clock cycle delays</li> <li>4: Sampling errors are randomly inserted with 0 or up to 0.5 destination clock cycle delays</li> </ul> |
| pulse_mode  | 0 to 3<br>Default: 0 | Selects the type of pulse presented to the input. <ul style="list-style-type: none"> <li>0: Single source domain clock cycle pulse transmitted to destination domain</li> <li>1: Rising transition detect transmitted as single cycle pulse in the destination domain</li> <li>2: Falling transition detect transmitted as single clock cycle pulse in the destination domain</li> <li>3: Toggle operation (rising or falling) is transmitted as a single-cycle pulse in the destination domain</li> </ul>                 |

**Table 1-3 Synthesis Implementations**

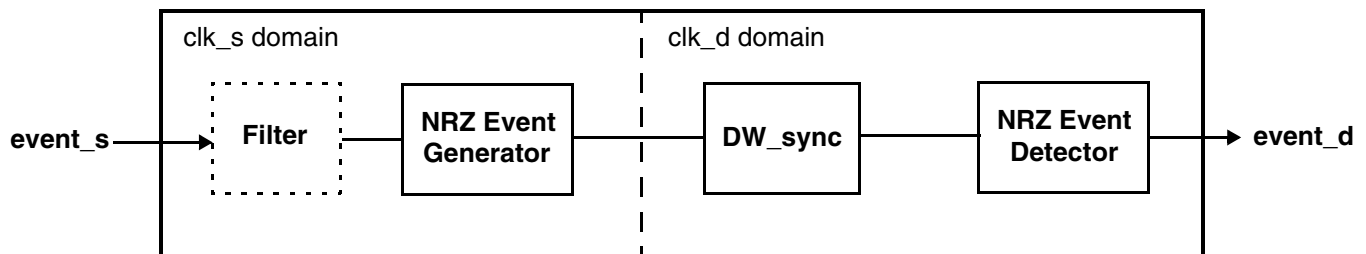
| Implementation Name | Function        | License Feature Required |
|---------------------|-----------------|--------------------------|
| rtl                 | Synthesis model | DesignWare               |

**Table 1-4 Simulation Models**

| Model                             | Function                                                                                                                    |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| DW03.DW_PULSE_SYNC_SIM            | Architectural name for VHDL simulation with missampling disabled.                                                           |
| DW03.DW_PULSE_SYNC_SIM_MS         | Architectural name for VHDL simulation with missampling enabled; see also " <a href="#">Simulation Methodology</a> " below. |
| dw/dw03/src/DW_pulse_sync_sim.vhd | VHDL simulation model source code (modeling RTL)                                                                            |
| dw/sim_ver/DW_data_sync_1c.v      | Verilog simulation model source code                                                                                        |

## Functional Description

The DW\_pulse\_sync synchronizer provides a method of passing event information from one clock domain to another. Using a Non-Return-to-Zero (NRZ) event generator (that is, a toggle register) triggered by the input, *event\_s*, in the source clock domain, the destination domain synchronizes the NRZ event signal (using an instance of DW\_sync) and then detects the NRZ event (that is, a toggle of the signal) and presents an active high output on *event\_d* for once cycle of *clk\_d* to signal the detected event.

**Figure 1-1 DW\_pulse\_sync Dual-clock Pulse Synchronizer Block Diagram**

The way in which the *event\_s* input triggers events depends on the value of the parameter, *pulse\_mode*.

**Table 1-5 pulse\_mode Parameter Trigger Method**

| <i>pulse_mode</i> | Trigger Method                                                                                                                             |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 0                 | Trigger an event for each clock cycle that <i>event_s</i> is high                                                                          |
| 1                 | Trigger an event for each clock cycle when <i>event_s</i> is high AND <i>event_s</i> was low for the previous clock (rising-edge detect)   |
| 2                 | Trigger an event for each clock cycle when <i>event_s</i> is low AND <i>event_s</i> was high for the previous clock (falling-edge detect)  |
| 3                 | Trigger an event for each clock cycle when <i>event_s</i> is different than it was for the previous clock (rising- or falling-edge detect) |

Although there are no required clock frequency relationships between the source and destination clocks, the rate of events that the synchronizer can reliably pass is restricted by the frequency of the destination clock. The time between NRZ events between the domains must not approach one clock period of *clk\_d* plus

some adequate setup time of a synchronization register. To be safe, it's best to allow at least two periods of `clk_d` between any two consecutive events.

This module safely transmits a pulse between two clock domains, provided the frequency of the transmitted pulse train does not exceed 1/2 the frequency of the receiving clock domain. For pulses transmitted from a low-speed domain to a high-speed domain, the pulse is not lost despite the speed ratio. For pulses transmitted between asynchronous domains where the transmitting clock is faster, even if they are nearly the same frequency, the transmitting domain must not transmit a pulse more often than once every three cycles of the low-speed domain; that is, the pulse train frequency must be less than 1/2 of the low-speed clock domain frequency. For higher-speed transmitting domains, the maximum pulse frequency must be determined according to the frequency of the receiving domain.

Because this block registers events across two time domains, if an odd number of events is passed to the source clock domain and then only the source domain reset is activated, the reset will cause an event in the receiving domain. The same is true if only the destination domain reset is applied.

To avoid unwanted events, both domains should be reset.

## Simulation Methodology

Because this component contains synchronizing devices, there are two methods available for simulation. One method is to use the simulation models that emulate the RTL model. Or, you can enable modeling of random skew between bits of signals traversing to and from each domain (called missampling).

To use the simulation models that emulate the RTL model, no special configuration is required.

To use missampling requires the following considerations:

- To enable missampling in Verilog simulations, define the macro `DW_MODEL_MISSAMPLES`:  

```
`define DW_MODEL_MISSAMPLES
```

If ``DW_MODEL_MISSAMPLES` is defined, the `verif_en` parameter comes into play to configure the simulation model as described by [Table 1-2](#) on page 2. If ``DW_MODEL_MISSAMPLES` is not defined, the Verilog simulation model behaves as if `verif_en` is set to 0.
- To enable missampling in VHDL simulations, a simulation architecture named `sim_ms` is provided. The parameter `verif_en` only has meaning when using `sim_ms`. That is, when the `sim` simulation architecture is used instead, the model behaves as though `verif_en` is set to 0.

## Suppressing Warning Messages During Verilog Simulation

The Verilog simulation model includes macros that allow you to suppress warning messages during simulation.

To suppress all warning messages for all DWBB components, define the `DW_SUPPRESS_WARN` macro in either of the following ways:

- Specify the Verilog preprocessing macro in Verilog code:  

```
`define DW_SUPPRESS_WARN
```
- Or, include a command line option to the simulator, such as:  

```
+define+DW_SUPPRESS_WARN
```

 (which is used for the Synopsys VCS simulator)

The warning messages for this model include the following:

- If values other than 1 or 0 are present on a clock port, the following message is displayed:

```
WARNING: <instance_path>.<clock_name>_monitor:
        at time = <timestamp>, Detected unknown value, x, on <clock_name> input.
```

To suppress only this warning message for all DWBB components, use the following macro:

- Define the DW\_DISABLE\_CLK\_MONITOR macro. You can define this macro in the following ways:
  - Specify the Verilog preprocessing macro in Verilog code:
 

```
`define DW_DISABLE_CLK_MONITOR
```
  - Or, include a command line option to the simulator, such as:
 

```
+define+DW_DISABLE_CLK_MONITOR
```

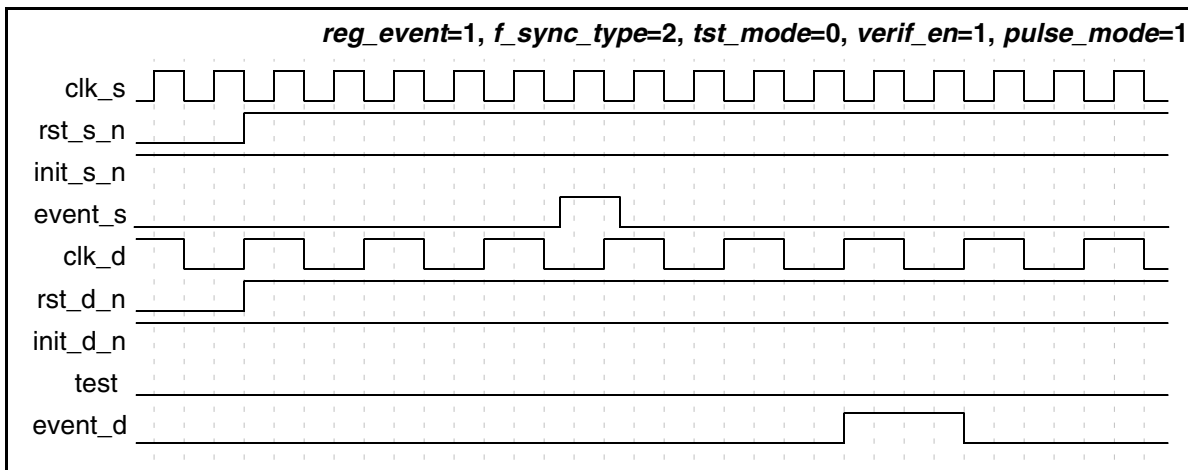
 (which is used for the Synopsys VCS simulator)

This message is also suppressed using the DW\_SUPPRESS\_WARN macro explained earlier.

## Timing Diagram

Figure 1-2 depicts pulse train timing from source domain to destination domain.

Figure 1-2 Post Reset Timing Diagram



## Related Topics

- [Memory – Registers Overview](#)
- [DesignWare Building Block IP User Guide](#)

## HDL Usage Through Component Instantiation - VHDL

```
library IEEE,DWARE,dw03;
use IEEE.std_logic_1164.all;
use DWARE.DW_Foundation_comp.all;

entity DW_pulse_sync_inst is
    generic (
        inst_reg_event : NATURAL := 1;
        inst_f_sync_type : NATURAL := 2;
        inst_tst_mode : NATURAL := 0;
        inst_verif_en : NATURAL := 1;
        inst_pulse_mode : NATURAL := 1
    );
    port (
        inst_clk_s : in std_logic;
        inst_rst_s_n : in std_logic;
        inst_init_s_n : in std_logic;
        inst_event_s : in std_logic;
        inst_clk_d : in std_logic;
        inst_rst_d_n : in std_logic;
        inst_init_d_n : in std_logic;
        inst_test : in std_logic;
        event_d_inst : out std_logic
    );
end DW_pulse_sync_inst;

architecture inst of DW_pulse_sync_inst is

begin

    -- Instance of DW_pulse_sync
    U1 : DW_pulse_sync
    generic map ( reg_event => inst_reg_event,
                  f_sync_type => inst_f_sync_type,
                  tst_mode => inst_tst_mode,
                  verif_en => inst_verif_en,
                  pulse_mode => inst_pulse_mode )
    port map ( clk_s => inst_clk_s,
               rst_s_n => inst_rst_s_n,
               init_s_n => inst_init_s_n,
               event_s => inst_event_s,
               clk_d => inst_clk_d,
               rst_d_n => inst_rst_d_n,
               init_d_n => inst_init_d_n,
               test => inst_test,
               event_d => event_d_inst );
```

```
end inst;
-- pragma translate_off
library DW03;
configuration DW_pulse_sync_inst_cfg_inst of DW_pulse_sync_inst is
  for inst
    end for; -- inst
end DW_pulse_sync_inst_cfg_inst;
-- pragma translate_on
```

## HDL Usage Through Component Instantiation - Verilog

```
module DW_pulse_sync_inst( inst_clk_s, inst_rst_s_n, inst_init_s_n, inst_event_s,
inst_clk_d,
                        inst_rst_d_n, inst_init_d_n, inst_test, event_d_inst );

parameter reg_event = 1;
parameter f_sync_type = 2;
parameter tst_mode = 0;
parameter verif_en = 1;
parameter pulse_mode = 1;

input inst_clk_s;
input inst_rst_s_n;
input inst_init_s_n;
input inst_event_s;
input inst_clk_d;
input inst_rst_d_n;
input inst_init_d_n;
input inst_test;
output event_d_inst;

// Instance of DW_pulse_sync
  DW_pulse_sync #(reg_event, f_sync_type, tst_mode, verif_en, pulse_mode)
    U1 ( .clk_s(inst_clk_s), .rst_s_n(inst_rst_s_n), .init_s_n(inst_init_s_n),
.event_s(inst_event_s), .clk_d(inst_clk_d), .rst_d_n(inst_rst_d_n),
.init_d_n(inst_init_d_n), .test(inst_test), .event_d(event_d_inst) );

endmodule
```

## Revision History

For notes about this release, see the [DesignWare Building Block IP Release Notes](#).

For lists of both known and fixed issues for this component, refer to the [STAR report](#).

For a version of this datasheet with visible change bars, click [here](#).

| Date           | Release       | Updates                                                                                                                                                                                                                                                                                                                                                                         |
|----------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| September 2024 | DWBB_202409.0 | ■ Updated date and version; no technical changes                                                                                                                                                                                                                                                                                                                                |
| December 2023  | DWBB_202312.0 | ■ Updated date and version; no technical changes                                                                                                                                                                                                                                                                                                                                |
| September 2021 | DWBB_202106.2 | ■ Updated the description of the <i>pulse_mode</i> parameter in <a href="#">Table 1-2</a> on page <a href="#">2</a> and <a href="#">Table 1-5</a> on page <a href="#">3</a>                                                                                                                                                                                                     |
| July 2020      | DWBB_201912.5 | <ul style="list-style-type: none"><li>■ Expanded the description of how to enable missampling during simulation in “<a href="#">Simulation Methodology</a>” on page <a href="#">4</a></li><li>■ Adjusted content and title of “<a href="#">Suppressing Warning Messages During Verilog Simulation</a>” on page <a href="#">4</a> and added the DW_SUPPRESS_WARN macro</li></ul> |
| October 2019   | DWBB_201903.5 | ■ Added the “Disabling Clock Monitor Messages” section                                                                                                                                                                                                                                                                                                                          |
| September 2018 | DWBB_201806.2 | <ul style="list-style-type: none"><li>■ Corrected typo in datasheet title</li><li>■ Added this Revision History table and the document links on this page</li></ul>                                                                                                                                                                                                             |

## Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

### Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

### Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

### Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

### Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

### Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.  
[www.synopsys.com](http://www.synopsys.com)

